

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:
«Компьютерные сети»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
«Анализ трафика компьютерных сетей утилитой Wireshark»

Выполнил:

Суханкулиев Мухаммет,
студент группы N3346


(подпись)

Проверил:

Ярошевский Дмитрий Сергеевич,
ведущий инженер, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Введение.....	4
1 Анализ трафика компьютерных сетей утилитой Wireshark	5
1.1 Анализ трафика утилиты ping	5
1.1.1 Описание выполненных действий	5
1.1.2 Ответы на вопросы задания.....	5
1.2 Анализ трафика утилиты tracert (traceroute)	8
1.2.1 Описание выполненных действий	8
1.2.2 Ответы на вопросы задания.....	8
1.3 Анализ HTTP-трафика	12
1.3.1 Описание выполненных действий	12
1.3.2 Анализ первичного посещения страницы.....	12
1.3.3 Анализ вторичного запроса (обновления) страницы	13
1.4 Анализ DNS-трафика	14
1.4.1 Описание выполненных действий	14
1.4.2 Ответы на вопросы задания.....	14
1.5 Анализ ARP-трафика.....	16
1.5.1 Описание выполненных действий	16
1.5.2 Ответы на вопросы задания.....	16
1.6 Анализ трафика утилиты nslookup.....	18
1.6.1 Описание выполненных действий	18
1.6.2 Ответы на вопросы задания.....	18
1.7 Анализ FTP-трафика	20
1.7.1 Описание выполненных действий	20
1.7.2 Ответы на вопросы задания.....	20
1.8 Анализ DHCP-трафика.....	23
1.8.1 Описание выполненных действий	23
1.8.2 Временная диаграмма обмена (DORA)	23
1.8.3 Ответы на вопросы задания.....	24
1.9 Анализ VK-трафика.....	26
1.9.1 Описание выполненных действий	26
1.9.2 Ответы на вопросы задания.....	27
Заключение.....	31
Список использованных источников.....	32

ВВЕДЕНИЕ

Цель работы – изучить структуру протокольных блоков данных, анализируя реальный трафик на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark.

В процессе выполнения учебно-исследовательской работы (УИР) необходимо:

- Освоить базовые навыки работы с анализатором трафика Wireshark: захват, фильтрация и анализ пакетов.
- Исследовать работу служебных протоколов ICMP, DNS и ARP на примере системных утилит ping, tracert и nslookup.
- Проанализировать структуру и принципы работы протоколов прикладного уровня, таких как HTTP и FTP, на примере взаимодействия с веб- и FTP-серверами.
- Изучить процесс получения IP-адреса с помощью протокола DHCP.
- Сравнить характеристики трафика различных типов данных (текст, аудио, видео) на примере мессенджера VK.

1 АНАЛИЗ ТРАФИКА КОМПЬЮТЕРНЫХ СЕТЕЙ УТИЛИТОЙ WIRESHARK

В качестве целевого узла для анализа во всех заданиях был выбран сайт www.sm-instruments.com. Данный выбор обусловлен соответствием требованиям методического указания:

- Доменное имя содержит инициалы (Суханкулиев Мухаммет -> S...M...).
- Сайт работает по протоколу HTTP, что позволяет анализировать незашифрованный трафик, как того требует задание 3.4.3.

1.1 Анализ трафика утилиты ping

1.1.1 Описание выполненных действий

Для анализа трафика утилиты ping была запущена командная строка и выполнены команды `ping -l <размер_пакета> www.sm-instruments.com` с различными значениями размера пакета: 1000, 2000, 3000, 4000, 5000 и 10000 байт. Одновременно с этим в программе Wireshark был запущен захват трафика с фильтром `icmp` для анализа передаваемых пакетов.

1.1.2 Ответы на вопросы задания

Вопрос 1: Имеет ли место фрагментация исходного пакета, какое поле на это указывает?

Да, фрагментация имеет место, когда общий размер IP-пакета превышает максимальный размер передаваемого блока (MTU) сети, который для Ethernet обычно составляет 1500 байт.

В моих экспериментах пакеты с размером данных 1000 байт (общий размер IP-пакета ~1028 байт) передавались без фрагментации. Пакеты с размером 2000, 3000 и 4000 байт были успешно доставлены, что означает, что они были фрагментированы для передачи и успешно собраны на принимающей стороне.

На наличие фрагментации указывают два поля в заголовке протокола IP (IPv4):

- Флаг «More Fragments»: Если этот флаг установлен в 1, это означает, что данный пакет является фрагментом и за ним последуют другие. У последнего фрагмента этот флаг равен 0.

– Поле «Fragment Offset»: Указывает на смещение данных этого фрагмента относительно начала исходного пакета. У первого фрагмента смещение равно 0.

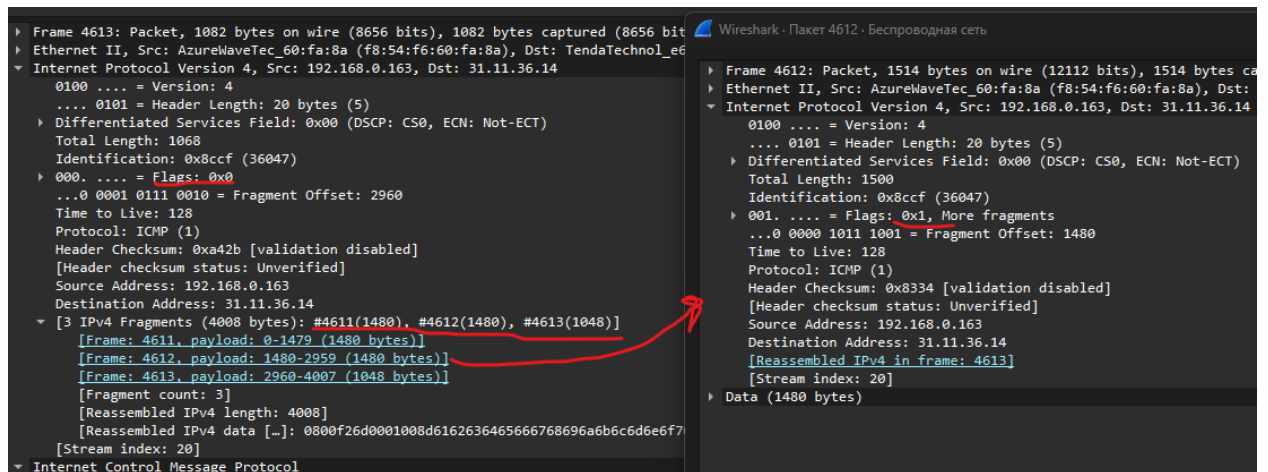


Рисунок 1 – Анализ фрагментов IP-пакета

Вопрос 2: Какая информация указывает, является ли фрагмент пакета последним или промежуточным?

Информация о том, является ли фрагмент промежуточным или последним, содержится в флаге More Fragments. Если флаг установлен (Set), фрагмент является промежуточным. Если он сброшен (Not set), а смещение (Fragment Offset) больше нуля, то фрагмент является последним в последовательности.

Вопрос 3: Чему равно количество фрагментов при передаче ping-пакетов?

Количество фрагментов зависит от размера данных и MTU сети (1500 байт). Расчет производится следующим образом: максимальный размер данных в одном IP-фрагменте составляет $1500 - 20$ (заголовок IP) = 1480 байт.

– ping -l 1000: Полезная нагрузка IP = 1000 (данные) + 8 (заголовок ICMP) = 1008 байт. Так как $1008 < 1480$, пакет не фрагментируется. Количество фрагментов = 1.

– ping -l 2000: Полезная нагрузка IP = 2008 байт. Первый фрагмент = 1480 байт. Остаток = $2008 - 1480 = 528$ байт. Второй фрагмент = 528 байт. Количество фрагментов = 2.

– ping -l 3000: Полезная нагрузка IP = 3008 байт. $3008 = 1480 + 1480 + 48$. Количество фрагментов = 3.

– ping -l 4000: Полезная нагрузка IP = 4008 байт. $4008 = 1480 + 1480 + 1048$. Количество фрагментов = 3.

Вопрос 4: Построить график, в котором на оси абсцисс находится размер_пакета, а по оси ординат – количество фрагментов, на которое был разделён каждый ping-пакет.

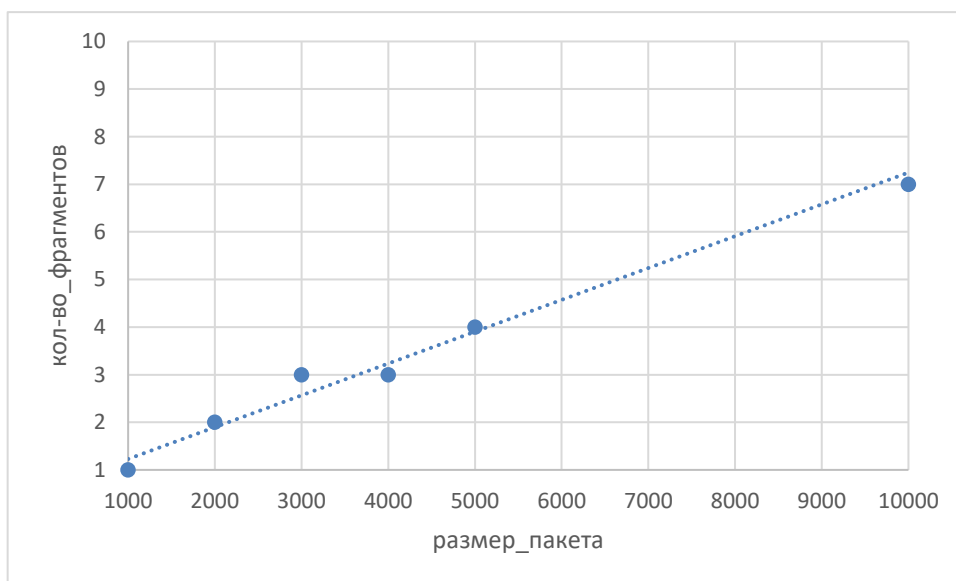


Рисунок 2 – График зависимости количества фрагментов от размера пакета

Вопрос 5: Как изменить поле TTL с помощью утилиты ping?

В операционной системе Windows для изменения поля TTL (Time-To-Live) в утилите ping используется ключ -i.

```
C:\Users\muham>ping -i 10 www.sm-instruments.com

Обмен пакетами с www.sm-instruments.com [31.11.36.14] с 32 байтами данных:
Ответ от 62.149.185.174: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 62.149.185.174: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 62.149.185.174: Превышен срок жизни (TTL) при передаче пакета.
Ответ от 62.149.185.174: Превышен срок жизни (TTL) при передаче пакета.

Статистика Ping для 31.11.36.14:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потеря)
```

Рисунок 3 – ICMP-запрос отправленный с установленным значением поля TTL = 10

Вопрос 6: Что содержится в поле данных ping-пакета?

В поле данных ICMP-пакетов, генерируемых утилитой ping в ОС Windows, по умолчанию содержится последовательность байт, соответствующая латинскому алфавиту, которая повторяется до заполнения заданного размера: abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz...

Frame 4684: Packet, 1042 bytes on wire (8336 bits), 1042 bytes captured on interface 0	0020	00 a3 00 00 59 cd 00 01 00 92 61 62 63 64 65 66	Y... abcde
Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: Azumi	0030	67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76	ghijklmn opqrstu
Internet Protocol Version 4, Src: 31.11.36.14, Dst: 192.168.0.163	0040	77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f	wabcdefg hijklmno
0100 = Version: 4	0050	70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68	pqrstuvw abcdefgh
.... 0101 = Header Length: 20 bytes (5)	0060	69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61	ijklmnop qrstuvwxyz
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)	0070	62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71	bcdefghi jklmnopq
Total Length: 1028	0080	72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a	rstuvwab cdefghij
Identification: 0x3f62 (16226)	0090	6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63	klmnopqr stuvwabc
000. = Flags: 0x0	00a0	64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73	defghijk lmnopqrs
...0 0000 0000 0000 = Fragment Offset: 0	00b0	74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c	tuvwxyz efghijkl
Time to Live: 53	00c0	6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65	mnopqrst uvwabcde
Protocol: ICMP (1)	00d0	66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75	fghijklm nopqrstu
Header Checksum: 0x3e33 [validation disabled]	00e0	76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e	vwabcdef ghijklmn
[Header checksum status: Unverified]	00f0	6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67	opqrstuv wabcdefg
Source Address: 31.11.36.14	0100	68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77	hijklmno pqrstuvw
Destination Address: 192.168.0.163	0110	61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70	abcdefgh ijklmnop
[Stream index: 20]	0120	71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69	qrstuvwxyz bcdefghi
Internet Control Message Protocol	0130	6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62	ijklmnopq rstuvwab
Type: Echo (ping) reply (0)	0140	63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72	cdefghij klmnopqr
Code: 0	0150	73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b	stuvwxyz defghijk
Checksum: 0x59cd [correct]	0160	6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64	lmnopqrs tuvwabcd
[Checksum Status: Good]	0170	65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74	efghijkl mnopqrst
Identifier (BE): 1 (0x0001)	0180	75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d	uvwabcde fghijklm
Identifier (LE): 256 (0x0100)	0190	6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66	nopqrstu vwabcdef
Sequence Number (BE): 146 (0x0092)	01a0	67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76	ghijklmn opqrstuv
Sequence Number (LE): 37376 (0x9200)	01b0	77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f	wabcdefg hijklmno
[Request frame: 4683]	01c0	70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68	pqrstuvw abcdefgh
[Response time: 64,973 ms]	01d0	69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61	ijklmnopq rstuvwab
Data (1000 bytes)	01e0	62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71	bcdefghi jklmnopq
Data [...]: 6162636465666768696a6b6c6d6e6f707172737475767761626364	01f0	72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 6a	rstuvwab cdefghij
[Length: 1000]	0200	6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63	klmnopqr stuvwabc
	0210	64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73	defghijk lmnopqrs
	0220	74 75 76 77 61 62 63 64 65 66 67 68 69 6a 6b 6c	tuvwxyz efghijkl
	0230	6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65	mnopqrst uvwabcde

Рисунок 4 – Содержимое поля данных ICMP-пакета (HEX и ASCII представления)

Следует отметить, что при попытке отправить пакеты размером 5000 и 10000 байт эхо-ответы не были получены. Это свидетельствует о наличии на промежуточном сетевом оборудовании или на целевом хосте системы безопасности, которая блокирует слишком большие ICMP-пакеты для предотвращения потенциальных атак.

1.2 Анализ трафика утилиты tracert (traceroute)

1.2.1 Описание выполненных действий

Для анализа трафика утилиты tracert была выполнена команда `tracert -d www.sm-instruments.com`. Ключ `-d` был использован для предотвращения разрешения IP-адресов в доменные имена. Захват и анализ трафика производился в программе Wireshark с использованием фильтра `icmp`.

1.2.2 Ответы на вопросы задания

Вопрос 1: Сколько байт содержится в заголовке IP? Сколько байт содержится в поле данных?

```

▶ Frame 204640: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 b
▶ Ethernet II, Src: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a), Dst: TendaTechno
▼ Internet Protocol Version 4, Src: 192.168.0.163, Dst: 31.11.36.14
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 92
    Identification: 0x8d0c (36108)
    ▶ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 12
    Protocol: ICMP (1)
    Header Checksum: 0x1d31 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.0.163
    Destination Address: 31.11.36.14
    [Stream index: 20]
▼ Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0xf738 [correct]
    [Checksum Status: Good]
    Identifier (BE): 1 (0x0001)
    Identifier (LE): 256 (0x0100)
    Sequence Number (BE): 198 (0x00c6)
    Sequence Number (LE): 50688 (0xc600)
    [Response frame: 204641]
▼ Data (64 bytes)
    Data: 0000000000000000000000000000000000000000000000000000000000000000
    [Length: 64]
```

Рисунок 5 – Заголовок IP составляет 20 байт, а поле данных - 64 байта

Согласно анализу захваченного трафика, стандартный заголовок IP-пакета (IPv4) без дополнительных опций составляет 20 байт, что соответствует теоретическим данным.

Однако размер поля данных в ICMP-запросах, генерируемых моей версией утилиты `tracert`, составил 64 байта, а не 32 байта, как это часто описывается в учебных материалах. Это говорит об особенностях реализации данной утилиты в моей операционной системе. На скриншоте видно поле `Data`.

Вопрос 2: Как и почему изменяется поле TTL в следующих друг за другом ICMP-пакетах `tracert`?

Утилита `tracert` работает, отправляя серии ICMP-запросов к конечному узлу, но намеренно манипулируя начальным значением поля TTL.

Как изменяется: `tracert` отправляет первую серию пакетов с TTL=1. Затем вторую серию с TTL=2, третью с TTL=3 и так далее, увеличивая TTL на единицу для каждого следующего хопа (маршрутизатора) на пути.

Почему изменяется: Это основной механизм работы утилиты. Каждый маршрутизатор, получая пакет, уменьшает его TTL на 1. Когда TTL становится равен 0, маршрутизатор отбрасывает пакет и отправляет источнику ICMP-сообщение об ошибке Time to live exceeded. tracert получает этот ответ, фиксирует IP-адрес маршрутизатора и переходит к следующему шагу, увеличив TTL. Процесс повторяется, пока пакет не достигнет конечной цели.

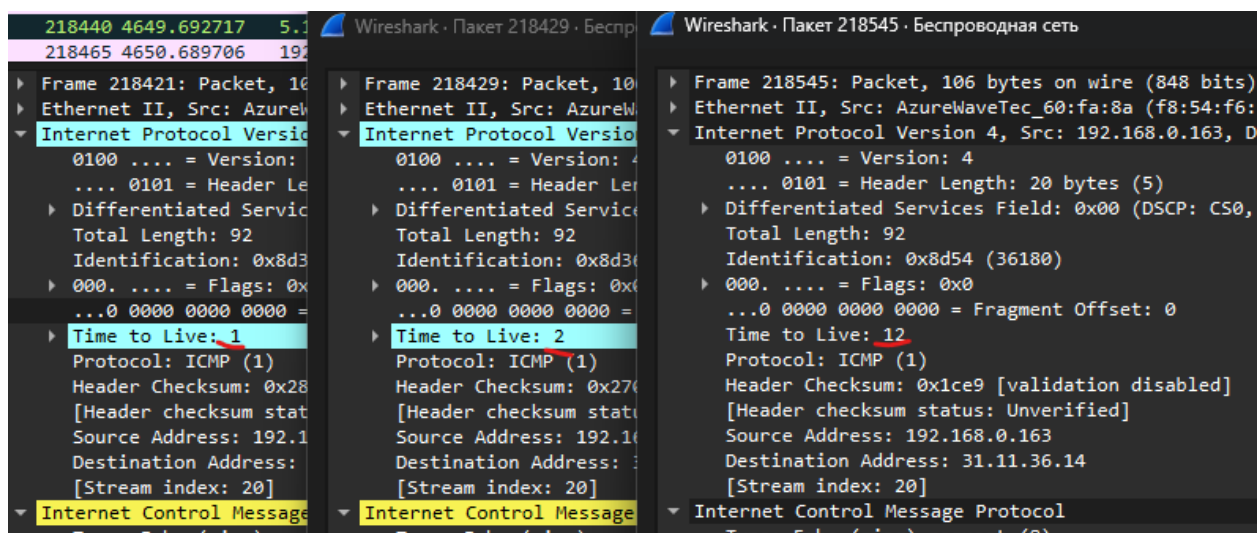


Рисунок 6 – tracert: TTL в последовательных исходящих ICMP-запросах

Вопрос 3: Чем отличаются ICMP-пакеты, генерируемые утилитой tracert, от ICMP-пакетов, генерируемых утилитой ping?

Несмотря на то, что обе утилиты могут использовать ICMP Echo Request, принципы их работы и ожидаемые ответы различаются:

- Начальное значение TTL: У ping начальное TTL высокое (в Windows по умолчанию 128) и не меняется. У tracert TTL намеренно низкое и последовательно увеличивается (1, 2, 3...).
- Ожидаемые ответы: ping ожидает получить только один тип ответа – Echo Reply (ответ на запрос). tracert же ожидает получить серию ICMP-сообщений об ошибке Time to live exceeded от промежуточных узлов и только в самом конце Echo Reply от целевого хоста.

Вопрос 4: Чем отличаются полученные пакеты «ICMP reply» от «ICMP error» и зачем нужны оба этих типа ответов?

- «ICMP error» (в данном случае Time to live exceeded, тип 11) – это служебное сообщение об ошибке. Оно информирует отправителя, что пакет не был доставлен, так как его «время жизни» истекло.

– «ICMP reply» (Echo reply, тип 0) – это штатный ответ на эхо-запрос. Он подтверждает, что пакет успешно достиг конечного адресата.

В утилите `tracert` необходимы оба типа ответов: сообщения `Time to live exceeded` конструктивно используются для определения IP-адресов промежуточных маршрутизаторов на пути, а финальный `Echo reply` служит сигналом того, что трассировка маршрута успешно завершена.

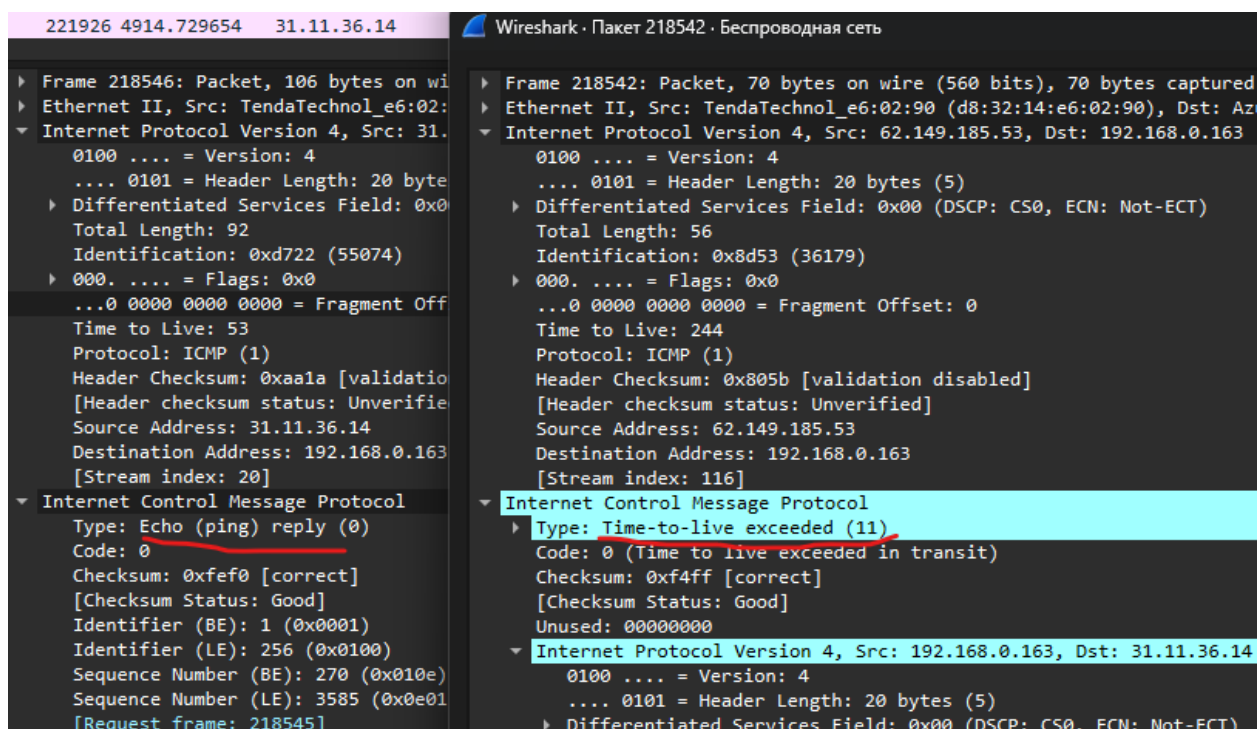


Рисунок 7 – Типы ответных пакетов в `tracert`

Вопрос 5: Что изменится в работе `tracert`, если убрать ключ `-d`? Какой дополнительный трафик при этом будет генерироваться?

Если убрать ключ `-d`, утилита `tracert` для каждого найденного IP-адреса промежуточного маршрутизатора и конечного узла будет пытаться определить его доменное имя.

Для этого будет генерироваться дополнительный трафик протокола DNS. В частности, `tracert` будет отправлять обратные DNS-запросы (`reverse DNS lookup`) для получения PTR-записей, которые сопоставляют IP-адреса с именами хостов.

224171	5211.771043	62.149.185.174	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
224172	5211.772018	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=306/12801, ttl=10 (no response found!)
224177	5211.831935	62.149.185.174	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
224189	5212.784362	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=307/13057, ttl=11 (no response found!)
224191	5212.844712	62.149.185.53	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224192	5212.845696	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=308/13313, ttl=11 (no response found!)
224193	5212.902987	62.149.185.53	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224194	5212.903770	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=309/13569, ttl=11 (no response found!)
224195	5212.953722	62.149.185.53	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224198	5213.915244	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=310/13825, ttl=12 (reply in 224199)
224199	5213.979518	31.11.36.14	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=310/13825, ttl=53 (request in 224198)
224200	5213.980493	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=311/14081, ttl=12 (reply in 224201)
224201	5214.044795	31.11.36.14	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=311/14081, ttl=53 (request in 224200)
224202	5214.045673	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=312/14337, ttl=12 (reply in 224203)
224203	5214.107074	31.11.36.14	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=312/14337, ttl=53 (request in 224202)
224205	5220.117321	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=313/14593, ttl=1 (no response found!)
224201	5220.132152	192.168.0.163	192.168.0.163	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
224232	5220.132628	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=314/14849, ttl=1 (no response found!)
224233	5220.136326	192.168.0.1	192.168.0.163	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
224234	5220.136761	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=315/15105, ttl=1 (no response found!)
224235	5220.140184	192.168.0.1	192.168.0.163	ICMP	134 Time-to-live exceeded (Time to live exceeded in transit)
224236	5220.140926	192.168.0.163	192.168.0.1	DNS	84 Standard query 0x0998 PTR 1.0.168.192.in-addr.arpa
224239	5220.149281	192.168.0.1	192.168.0.163	DNS	143 Standard query response 0x0998 No such name PTR 1.0.168.192.in-addr.arpa SOA localhost
224265	5223.653984	192.168.0.163	192.168.0.1	DNS	94 Standard query 0xae08 A p2p-st01.discovery.steamserver.net
224266	5223.659152	192.168.0.1	192.168.0.163	DNS	142 Standard query response 0xae08 A p2p-st01.discovery.steamserver.net A 162.254.198.44 A 162.254.198.46 A 162.254.198.104
224267	5226.085066	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=316/15361, ttl=2 (no response found!)
224268	5226.093535	5.19.0.126	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224269	5226.094383	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=317/15617, ttl=2 (no response found!)
224270	5226.149567	5.19.0.122	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224271	5226.150381	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=318/15873, ttl=2 (no response found!)
224272	5226.193787	5.19.0.126	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224273	5226.194822	192.168.0.163	192.168.0.1	DNS	83 Standard query 0x784e PTR 126.0.19.5.in-addr.arpa
224274	5226.203629	192.168.0.1	192.168.0.163	DNS	140 Standard query response 0x784e PTR 126.0.19.5.in-addr.arpa PTR 5x19x0x126.static-business.spb.ertelecom.ru
224297	5231.748535	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=319/16129, ttl=3 (no response found!)
224298	5231.754873	5.19.0.201	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224299	5231.755730	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=320/16385, ttl=3 (no response found!)
224300	5231.762184	5.19.0.201	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
224301	5231.763059	192.168.0.163	31.11.36.14	ICMP	106 Echo (ping) request id=0x0001, seq=321/16641, ttl=3 (no response found!)

Рисунок 8 – tracer: без ключа -d (внизу) появляются дополнительные DNS-запросы

1.3 Анализ HTTP-трафика

1.3.1 Описание выполненных действий

Для анализа HTTP-трафика было смоделировано два сценария взаимодействия с сайтом <http://www.sm-instruments.com>.

- Первичное посещение: После полной очистки кэша браузера была выполнена загрузка главной страницы сайта.
- Вторичное посещение: Страница была обновлена с помощью стандартной функции браузера (клавиша F5).

Оба сценария были зафиксированы с помощью анализатора трафика Wireshark с использованием фильтра http.

1.3.2 Анализ первичного посещения страницы

При первичном посещении сайта, после очистки кэша, браузер запрашивает все необходимые ресурсы для отображения страницы.

- Запрос HTML-документа: Сначала отправляется GET-запрос для получения основного HTML-кода страницы.
- Ответ сервера 200 OK: Сервер возвращает полный HTML-документ с кодом 200 OK. При этом в заголовках ответа сервер указывает директивы Cache-Control: no-cache и Pragma: no-cache, запрещая кэшировать данный HTML-файл.

– Запрос статических ресурсов: После получения HTML браузер запрашивает статические файлы (изображения, скрипты). В ответах на них сервер присылает заголовок Last-Modified, который браузер сохраняет для будущих проверок.

No.	Time	Source	Destination	Protocol	Length	Info
52	2.148264	192.168.0.163	31.11.36.14	HTTP	574	GET /en/ HTTP/1.1
58	2.446157	31.11.36.14	192.168.0.163	HTTP	134	HTTP/1.1 200 OK (text/html)
60	2.520050	192.168.0.163	31.11.36.14	HTTP	559	GET /plugins/editors/jckeditor/typography/typography.php HTTP/1.1
63	2.527133	192.168.0.163	31.11.36.14	HTTP	546	GET /cache/widgetkit/widgetkit-8c4942b1.css HTTP/1.1
66	2.529411	192.168.0.163	209.85.233.95	HTTP	414	GET /ajax/libs/jquery/1.7.0/jquery.min.js HTTP/1.1
96	2.573227	209.85.233.95	192.168.0.163	HTTP	535	HTTP/1.1 200 OK (text/javascript)


```

Frame 58: Packet, 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0
Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4, Src: 31.11.36.14, Dst: 192.168.0.163
Transmission Control Protocol, Src Port: 80, Dst Port: 37848, Seq: 6193, Ack: 969, Len: 80
[5 Reassembled TCP Segments (5920 bytes): #54(1460), #55(1460), #56(1460), #57(1460), #58(80)]
Hypertext Transfer Protocol, has 2 chunks (including last chunk)
  HTTP/1.1 200 OK\r\n
    Response Version: HTTP/1.1
    Status Code: 200
    [Status Code Description: OK]
    Response Phrase: OK
    Server: aruba-proxy\r\n
    Date: Thu, 09 Oct 2025 19:38:43 GMT\r\n
    Content-Type: text/html; charset=utf-8\r\n
    Transfer-Encoding: chunked\r\n
    Connection: keep-alive\r\n
    Vary: Accept-Encoding\r\n
    P3P: CP="NOI ADM DEV PSAi COM NAV OUR OTRo STP IND DEM"\r\n
    Cache-Control: no-cache\r\n
    Pragma: no-cache\r\n
    Set-Cookie: 734955dcac07efc356dd5e79dfbef238=en-GB; expires=Fri, 09-Oct-2026 19:38:43 GMT; Max-Age=31536000; path=/\r\n
    X-ServerName: webx.aruba.it\r\n
    Alt-Svc: h3=":443"; ma=86400\r\n
    Content-Encoding: gzip\r\n
    \r\n
    [Request in frame: 52]
    [Time since request: 297.893000 milliseconds]
    [Request URI: /en/]
    [Full request URI: http://www.sm-instruments.com/en/]
  HTTP chunked response
    Content-encoded entity body (gzip): 5415 bytes -> 20071 bytes
    File Data: 20071 bytes
  Line-based text data: text/html (387 lines)

```

Рисунок 9 – Первичная загрузка: ответ 200 ОК с запретом кэширования HTML

1.3.3 Анализ вторичного запроса (обновления) страницы

При обновлении страницы наблюдается разное поведение для разных типов контента, что обусловлено политиками кэширования, заданными сервером.

– Запрос HTML-документа: Поскольку кэширование HTML-файла было запрещено сервером, браузер вынужден запросить его полную версию заново. В запросе присутствует заголовок Cache-Control: max-age=0 (требование свежей версии). Сервер снова отвечает 200 ОК и присылает полный HTML-код. Механизм ответа 304 Not Modified для этого ресурса не используется.

No.	Time	Source	Destination	Protocol	Length	Info
37	4.754401	192.168.0.163	31.11.36.14	HTTP	849	GET /templates/aries_template/images/menu_dropdown_level1_parent.png HTTP/1.1
38	4.826455	31.11.36.14	192.168.0.163	HTTP	454	HTTP/1.1 200 OK (PNG)
46	5.221674	192.168.0.163	31.11.36.14	HTTP	837	GET /en/ HTTP/1.1
55	5.402750	31.11.36.14	192.168.0.163	HTTP	134	HTTP/1.1 200 OK (text/html)
57	5.471836	192.168.0.163	31.11.36.14	HTTP	886	GET /media/widgetkit/widgets/lightbox/js/lightbox.js?_1760038762769 HTTP/1.1
58	5.477525	192.168.0.163	31.11.36.14	HTTP	914	GET /media/widgetkit/widgets/mediaplayer/mediaelement/mediaelement-and-player

Frame 55: Packet, 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits) on interface \Device\NPF_{A271AD40-B4FE-4657-86E0-9591C47CF3DE}, id 0
Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4, Src: 31.11.36.14, Dst: 192.168.0.163
Transmission Control Protocol, Src Port: 80, Dst Port: 37848, Seq: 6241, Ack: 1579, Len: 80
[5 Reassembled TCP Segments (5920 bytes): #51(1460), #52(1460), #53(1460), #54(1460), #55(80)]
Hypertext Transfer Protocol, has 2 chunks (including last chunk)

HTTP/1.1 200 OK\r\n
Response Version: HTTP/1.1
Status Code: 200
[Status Code Description: OK]
Response Phrase: OK
Server: aruba-proxy\r\n
Date: Thu, 09 Oct 2025 19:39:22 GMT\r\n
Content-Type: text/html; charset=utf-8\r\n
Transfer-Encoding: chunked\r\n
Connection: keep-alive\r\n
Vary: Accept-Encoding\r\n
P3P: CP="NOI ADM DEV PSAi COM NAV OUR OTRo STP IND DEM"\r\n
Cache-Control: no-cache\r\n
Pragma: no-cache\r\n
Set-Cookie: /34955dcac07efc356dd5e79dfbef238=en-GB; expires=Fri, 09-Oct-2026 19:39:22 GMT; Max-Age=31536000; path=/\r\n
X-ServerName: webx.aruba.it\r\n
Alt-Svc: h3=":443"; ma=86400\r\n
Content-Encoding: gzip\r\n
\r\n
[Request in frame: 46]
[Time since request: 181.076000 milliseconds]
[Request URI: /en/]
[Full request URI: http://www.sm-instruments.com/en/]

HTTP chunked response
Content-encoded entity body (gzip): 5415 bytes -> 20071 bytes
File Data: 20071 bytes

Line-based text data: text/html (387 lines)

Рисунок 10 – Обновление страницы: повторный запрос HTML с ответом 200 OK

1.4 Анализ DNS-трафика

1.4.1 Описание выполненных действий

Для анализа трафика протокола DNS была проведена подготовка: очищен системный кэш DNS с помощью команды `ipconfig /flushdns` и очищен кэш браузера. После этого был запущен захват трафика в Wireshark с фильтром `dns` и выполнен переход на сайт www.sm-instruments.com.

1.4.2 Ответы на вопросы задания

Вопрос 1: Почему адрес, на который отправлен DNS-запрос, не совпадает с адресом посещаемого сайта?

DNS-запрос отправляется на IP-адрес DNS-сервера, а не на IP-адрес целевого сайта. Как видно из лога, все запросы от моего компьютера (192.168.0.163) отправляются на адрес 192.168.0.1, который является адресом моего локального DNS-сервера (роутера). Например, в пакете №441 мой компьютер спрашивает у сервера 192.168.0.1 IP-адрес для имени www.sm-instruments.com, и только в ответе (пакет №458) получает искомый IP 31.11.36.14.

65	2.471839	192.168.0.1	192.168.0.163	DNS	156 Standard query response 0x0873 A r.msftstatic.com CNAME r.msftstatic-com.a-0016.a-msedge.net CNAME a-0016.a-msedge.net A 204.79.197.219
441	4.961492	192.168.0.163	192.168.0.1	DNS	82 Standard query 0xf996 A www.sm-instruments.com
442	4.978209	192.168.0.163	192.168.0.1	DNS	75 Standard query 0x5fb6 A apis.google.com
443	4.978597	192.168.0.163	192.168.0.1	DNS	79 Standard query 0x4f31 A ajax.googleapis.com
446	4.983553	192.168.0.1	192.168.0.163	DNS	136 Standard query response 0x5fb6 A apis.google.com CNAME plus-1.google.com CNAME wide-plus-1.google.com A 74.125.205.196
447	4.985642	192.168.0.1	192.168.0.163	DNS	95 Standard query response 0x4f31 A ajax.googleapis.com A 209.85.233.95
458	5.029722	192.168.0.1	192.168.0.163	DNS	98 Standard query response 0xf996 A www.sm-instruments.com A 31.11.36.14
1085	6.766370	192.168.0.163	192.168.0.1	FWC	70 Standard query 0x4f31 A ajax.googleapis.com

Рисунок 11 – Процесс разрешения DNS-имени

Вопрос 2: Какие бывают типы DNS-запросов?

Существует множество типов DNS-записей, которые может запрашивать клиент. В ходе анализа трафика при загрузке сайта были зафиксированы следующие типы запросов:

- А (Address): Самый распространенный тип запроса для получения IPv4-адреса, сопоставленного с доменным именем.

- AAAA (Quad A): Запрос для получения IPv6-адреса.

Также существуют и другие важные типы DNS-записей, такие как:

- CNAME (Canonical Name): Указывает, что домен является псевдонимом для другого ("канонического") домена.

- MX (Mail Exchange): Определяет почтовые серверы для домена.

- NS (Name Server): Указывает на авторитативные DNS-серверы, ответственные за доменную зону.

В захваченном трафике были зафиксированы DNS-запросы типа А (для получения IPv4-адреса). Однако в ответах сервера были обнаружены и другие типы записей, в частности CNAME (Canonical Name). Например, в пакете №30 в ответ на А-запрос для browser.events.data.msn.com сервер вернул цепочку из нескольких CNAME-записей, указывающих на реальное имя хоста, для которого уже была предоставлена А-запись. Это демонстрирует использование псевдонимов в системе DNS.

No	Time	Source	Destination	Protocol	Length	Info
22	2.431589	192.168.0.163	192.168.0.1	DNS	87	Standard query 0xa29e A browser.events.data.msn.com
30	2.447358	192.168.0.1	192.168.0.163	DNS	214	Standard query response 0xa29e A browser.events.data.msn.com CNAME global.asimov.events.data.trafficmanager.net CNAME onedcolprduw99.westus.cloudapp.azure.com A 20.189.173.10
32	3.031053	192.168.0.163	192.168.0.1	DNS	70	Standard query 0x23af A browser.events.data.msn.com
Frame 30: Packet, 214 bytes on wire (1732 bits), 214 bytes captured (1732 bits) on Interface \Device\NPF_{A271A04D-BAFE-4657-86E0-9591C47CF30E}, Id 0 Ethernet II, Src: TendaTechnology_88:63:10:12:14:ed:02:00, Dst: AzureWaveTec_68:fa:8a: (f8:54:f6:60:fa:8a) Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.163 User Datagram Protocol, Src Port: 53, Dst Port: 5320 Domain Name System (response) Transaction ID: 0xa29e Flags: 0x10 Standard query response, No error Questions: 1 Answer RRs: 3 Authority RRs: 0 Additional RRs: 0 Queries Answers browser.events.data.msn.com: type CNAME, class IN, cname global.asimov.events.data.trafficmanager.net global.asimov.events.data.trafficmanager.net: type CNAME, class IN, cname onedcolprduw99.westus.cloudapp.azure.com onedcolprduw99.westus.cloudapp.azure.com: type A, class IN, addr 20.189.173.10 [Request ID: 22] [Time: 15.859000 milliseconds]						

Рисунок 12 – Пример ответа DNS с CNAME-записями

Вопрос 3: В какой ситуации нужно выполнять независимые DNS-запросы для получения содержащихся на сайте изображений?

Необходимость в выполнении независимых DNS-запросов для изображений (или любых других ресурсов, таких как скрипты, стили, видео) возникает в том случае, если эти ресурсы размещены на другом домене (или поддомене).

Мой анализ трафика это наглядно подтвердил. После загрузки основной страницы с www.sm-instruments.com, браузер обнаружил, что для ее полноценной работы требуются скрипты с других серверов. Как видно из лога, были инициированы отдельные DNS-запросы для определения IP-адресов хостов:

- apis.google.com (пакет №442)
- ajax.googleapis.com (пакет №443)
- www.google-analytics.com (пакет №1088)
- platform.twitter.com (пакет №1089)

Это доказывает, что для загрузки каждого ресурса с уникального доменного имени требуется предварительное разрешение этого имени через DNS.

1.5 Анализ ARP-трафика

1.5.1 Описание выполненных действий

Для анализа трафика протокола ARP была проведена подготовка: с помощью команды `netsh interface ip delete arpcache` был очищен системный ARP-кэш, что было проверено командой `arp -a`. После этого был запущен захват трафика в Wireshark с фильтром `arp` и выполнен переход на сайт www.sm-instruments.com для инициации сетевой активности.

1.5.2 Ответы на вопросы задания

Вопрос 1: Какие MAC-адреса присутствуют в захваченных пакетах ARP-протокола? Что означают эти адреса? Какие устройства они идентифицируют?

В захваченном трафике был зафиксирован ARP-обмен, инициированный моим роутером.

ARP-запрос (Request, пакет №973):

- Source MAC: (MAC-адрес роутера).
- Destination MAC: (MAC-адрес компьютера).

Этот пакет является запросом от роутера, который пытается определить MAC-адрес моего компьютера.

ARP-ответ (Reply, пакет №974):

- Source MAC: (MAC-адрес компьютера).
- Destination MAC: (MAC-адрес роутера).

Это ответ от моего компьютера, который сообщает роутеру свой MAC-адрес.

Таким образом, эти MAC-адреса идентифицируют сетевой интерфейс моего компьютера и сетевой интерфейс моего роутера.

The image shows a Wireshark packet capture of an ARP exchange. The top part shows two ARP packets: one from the router (TendaTechnol_e6:02:90) to the computer (AzureWaveTec_60:fa:8a) asking for the MAC address of 192.168.0.163, and a reply from the computer. The bottom part shows the details of the reply packet, including the Ethernet II header with source and destination MAC addresses, and the ARP payload showing the sender and target IP and MAC addresses.

Рисунок 13 – ARP-обмен между роутером и компьютером

Вопрос 2: Какие MAC-адреса присутствуют в захваченных HTTP-пакетах и что означают эти адреса? Какие устройства они идентифицируют?

В заголовках канального уровня (Ethernet II) всех исходящих HTTP-пакетов, отправленных моим компьютером, присутствуют следующие MAC-адреса:

- Source MAC: (MAC-адрес компьютера).
- Destination MAC: (MAC-адрес роутера).

MAC-адреса используются для доставки кадра в пределах локальной сети. Чтобы отправить HTTP-запрос в интернет, мой компьютер адресует его на MAC-адрес своего шлюза по умолчанию. MAC-адрес конечного веб-сервера в этом кадре не используется.

The image shows a Wireshark packet capture of an HTTP GET request. The top part shows a table of captured packets with columns for No., Time, Source, Destination, Protocol, Length, and Info. The selected packet is a GET request from 192.168.0.163 to 31.11.36.14. The bottom part shows the details of the packet, including the Ethernet II header with source and destination MAC addresses, the IPv4 header, the TCP header, and the HTTP GET request body.

Рисунок 14 – MAC-адреса в заголовке Ethernet HTTP-пакета

Вопрос 3: Для чего ARP-запрос содержит IP-адрес источника?

ARP-запрос содержит IP-адрес источника (Sender IP address) для того, чтобы узел, получивший запрос, знал, кому отправлять ARP-ответ.

В моем примере (пакет №973) роутер указал свой IP-адрес 192.168.0.1 в качестве Sender IP. Мой компьютер, получив этот запрос, прочитал этот IP-адрес и MAC-адрес отправителя и смог сформировать адресный (unicast) ARP-ответ, направив его точно на MAC-адрес роутера. Без этой информации мой компьютер не знал бы, какому именно из устройств в сети предназначен его ответ.

1.6 Анализ трафика утилиты nslookup

1.6.1 Описание выполненных действий

Для анализа трафика утилиты nslookup было проведено два эксперимента с использованием фильтра dns в Wireshark.

- Была выполнена команда nslookup www.sm-instruments.com для получения A-записи домена.
- Была выполнена команда nslookup -type=NS sm-instruments.com для получения NS-записей, определяющих авторитативные серверы домена.

1.6.2 Ответы на вопросы задания

Вопрос 1: Чем различается трасса трафика в п.2 и п.4, указанных выше?

Трассы трафика различаются типом запрашиваемой DNS-записи и, как следствие, содержанием ответа.

- В первом эксперименте (nslookup www.sm-instruments.com) был отправлен DNS-запрос на получение записи типа A (Address). Цель этого запроса – узнать IPv4-адрес хоста.
- Во втором эксперименте (nslookup -type=NS sm-instruments.com) был отправлен DNS-запрос на получение записей типа NS (Name Server). Цель этого запроса – узнать доменные имена серверов, которые несут ответственность за всю доменную зону sm-instruments.com.

55	3.943029	192.168.0.163	192.168.0.1	DNS	82 Standard query 0x0002 A www.sm-instruments.com
56	3.947806	192.168.0.1	192.168.0.163	DNS	98 Standard query response 0x0002 A www.sm-instruments.com A 31.11.36.14
57	3.950126	192.168.0.163	192.168.0.1	DNS	82 Standard query 0x0003 AAAA www.sm-instruments.com
58	4.011988	192.168.0.1	192.168.0.163	DNS	144 Standard query response 0x0003 AAAA www.sm-instruments.com SOA dns.technorail.

Frame 56: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0

Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)

Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.163

User Datagram Protocol, Src Port: 53, Dst Port: 49666

Domain Name System (response)

Transaction ID: 0x0002

Flags: 0x8180 Standard query response, No error

Questions: 1

Answer RRs: 1

Authority RRs: 0

Additional RRs: 0

Queries

Answers

- www.sm-instruments.com: type A, class IN, addr 31.11.36.14
 - Name: www.sm-instruments.com
 - Type: A (1) (Host Address)
 - Class: IN (0x0001)
 - Time to live: 21600 (6 hours)
 - Data length: 4
 - Address: 31.11.36.14

[Request In: 55]

[Time: 4.777000 milliseconds]

8	2.205631	192.168.0.1	192.168.0.163	DNS	143 Standard query response 0x0001 No such name PTR 1.0.168.192.in-addr.arpa SOA localhost
9	2.206902	192.168.0.163	192.168.0.1	DNS	78 Standard query 0x0002 NS sm-instruments.com
10	2.270191	192.168.0.1	192.168.0.163	DNS	187 Standard query response 0x0002 NS sm-instruments.com NS dns4.arubadns.cz NS dns3.arubadns.net NS

Frame 10: Packet, 187 bytes on wire (1496 bits), 187 bytes captured (1496 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0

Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)

Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.163

User Datagram Protocol, Src Port: 53, Dst Port: 52141

Domain Name System (response)

Transaction ID: 0x0002

Flags: 0x8180 Standard query response, No error

Questions: 1

Answer RRs: 4

Authority RRs: 0

Additional RRs: 0

Queries

- sm-instruments.com: type NS, class IN
 - Name: sm-instruments.com
 - [Name Length: 18]
 - [Label Count: 2]
 - Type: NS (2) (authoritative Name Server)
 - Class: IN (0x0001)

Answers

- sm-instruments.com: type NS, class IN, ns dns4.arubadns.cz
 - Name: sm-instruments.com
 - Type: NS (2) (authoritative Name Server)
 - Class: IN (0x0001)
 - Time to live: 21600 (6 hours)
 - Data length: 18
 - Name Server: dns4.arubadns.cz
- sm-instruments.com: type NS, class IN, ns dns3.arubadns.net
 - Name: sm-instruments.com
 - Type: NS (2) (authoritative Name Server)
 - Class: IN (0x0001)
 - Time to live: 21600 (6 hours)
 - Data length: 19
 - Name Server: dns3.arubadns.net
- sm-instruments.com: type NS, class IN, ns dns.technorail.com
 - Name: sm-instruments.com
 - Type: NS (2) (authoritative Name Server)
 - Class: IN (0x0001)
 - Time to live: 21600 (6 hours)
 - Data length: 19
 - Name Server: dns.technorail.com

Рисунок 15 – Сравнение DNS-запросов типа A и NS

Вопрос 2: Что содержится в поле «Answers» DNS-ответа?

Поле «Answers» содержит запрошенные DNS-записи (Resource Records). Его содержимое напрямую зависит от типа запроса:

В ответе на А-запрос поле «Answers» содержало одну А-запись, включающую IP-адрес хоста www.sm-instruments.com (31.11.36.14), его класс (IN) и время жизни записи (TTL).

В ответе на NS-запрос поле «Answers» содержало несколько NS-записей. Каждая такая запись содержала доменное имя одного из авторитативных серверов для зоны sm-instruments.com (например, ns1.aruba.it, ns2.aruba.it).

Вопрос 3: Каковы имена серверов, возвращающих авторитативный (authoritative) отклик?

Ответ DNS-сервера является авторитативным, если получен напрямую от сервера, ответственного за доменную зону. Ответы от кэширующих серверов (например, моего роутера 192.168.0.1) являются неавторитативными. Анализ трафика подтвердил это: в полученных DNS-ответах флаг Authoritative был сброшен (см. Рисунок 16), так как ответ был предоставлен локальным кэширующим сервером.

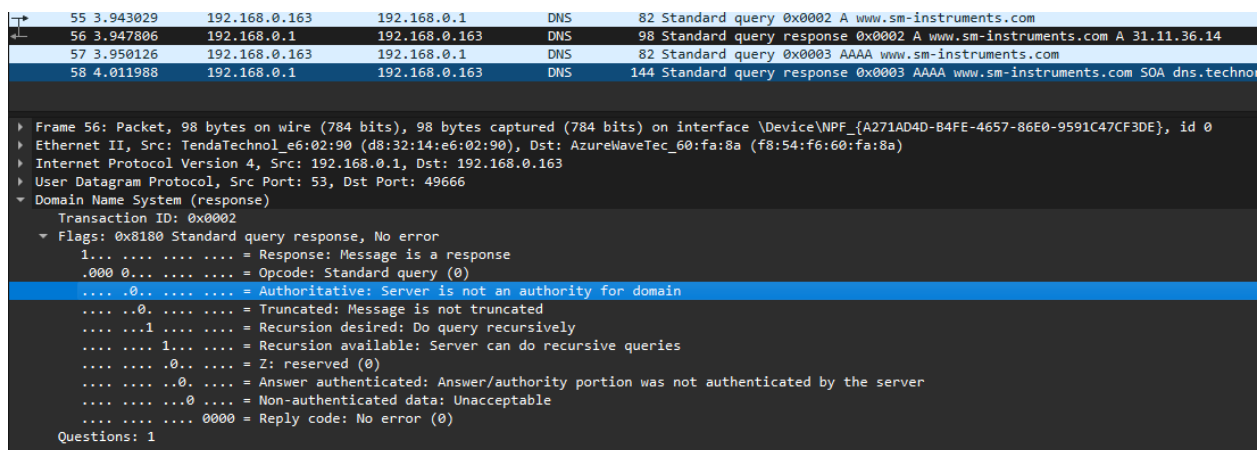


Рисунок 16 – Неавторитативный ответ DNS-сервера

1.7 Анализ FTP-трафика

1.7.1 Описание выполненных действий

В связи с тем, что найти публичный FTP-сервер, соответствующий требованиям варианта, не представилось возможным, а современные браузеры прекратили поддержку протокола FTP, для выполнения задания был использован публичный FTP-сервер ftp.gnu.org и FTP-клиент FileZilla.

Был произведен захват трафика в Wireshark с фильтром `ftp || ftp-data` в процессе анонимного подключения к серверу и скачивания текстового файла README.

1.7.2 Ответы на вопросы задания

Вопрос 1: Сколько байт данных содержится в пакете FTP-DATA?

Размер данных в пакете FTP-DATA зависит от максимального размера сегмента (MSS). В моем эксперименте для передачи небольшого объема данных (например, листинга каталога или маленького файла) использовался TCP-сегмент размером 132 байта (см. Рисунок 17). Для более крупных файлов размер сегментов стремился бы к значению MSS (Maximum Segment Size), которое для Ethernet-сетей обычно составляет 1460 байт.

125	8.359852	209.51.188.20	192.168.0.163	FTP	104 Response: 227 Entering Passive Mode (209,51,188,20,99,31).
126	8.368144	192.168.0.163	209.51.188.20	FTP	60 Request: LIST
133	8.455030	209.51.188.20	192.168.0.163	FTP	93 Response: 150 Here comes the directory listing.
134	8.555030	209.51.188.20	192.168.0.163	FTP-DATA	186 FTP Data: 132 bytes (PASV) (LIST)
140	8.826781	209.51.188.20	192.168.0.163	FTP	78 Response: 226 Directory send OK.
144	9.927674	192.168.0.163	209.51.188.20	FTP	68 Request: CWD /mirrors
146	10.066673	209.51.188.20	192.168.0.163	FTP	91 Response: 250 Directory successfully changed.
147	10.066885	192.168.0.163	209.51.188.20	FTP	59 Request: PWD
151	10.217684	209.51.188.20	192.168.0.163	FTP	95 Response: 257 "/mirrors" is the current directory


```

Frame 134: Packet, 186 bytes on wire (1488 bits), 186 bytes captured (1488 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0
Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4, Src: 209.51.188.20, Dst: 192.168.0.163
Transmission Control Protocol, Src Port: 25375, Dst Port: 55830, Seq: 1, Ack: 1, Len: 132
  Source Port: 25375
  Destination Port: 55830
  [Stream index: 13]
  [Stream Packet Number: 4]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 132]
    Sequence Number: 1 (relative sequence number)
    Sequence Number (raw): 29665308
    [Next Sequence Number: 133 (relative sequence number)]

```

Рисунок 17 – Длина TCP-сегмента в пакете FTP-DATA

Вопрос 2: Как выбирается порт транспортного уровня, который используется для передачи FTP-пакетов?

Протокол FTP использует два типа соединений, работающих на разных портах транспортного уровня:

- Управляющее соединение (FTP): Это основное соединение для передачи команд. Клиент всегда иницирует его, подключаясь к стандартному порту 21 на сервере. Это соединение остается активным в течение всей сессии.

- Соединение для передачи данных (FTP-DATA): Это временное соединение, создаваемое для передачи содержимого файла. В современном пассивном режиме FTP порт для этого соединения выбирается динамически, что и было зафиксировано в ходе эксперимента:

- Сначала клиент по управляющему соединению отправляет команду PASV.
- В ответ сервер присылает сообщение с кодом 227 Entering Passive Mode. В этом сообщении он указывает IP-адрес и номер порта, который он специально открыл для клиента.
- Как видно из анализа пакета №88 (см. Рисунок 18), сервер ftp.gnu.org прислал ответ: 227 Entering Passive Mode (209,51,188,20,83,100).
- Это означает, что для скачивания файла клиент должен установить новое TCP-соединение с IP-адресом 209.51.188.20 на порт, вычисляемый по формуле $(83 * 256) + 100 = 21348$.

Таким образом, порт для передачи команд является статическим (21), а порт для передачи данных – динамическим и сообщается сервером клиенту в ответ на команду PASV.

```

85 4.409401 192.168.0.163 209.51.188.20 FTP 60 Request: PASV
88 4.554643 209.51.188.20 192.168.0.163 FTP 105 Response: 227 Entering Passive Mode (209,51,188,20,83,100).
89 4.555015 192.168.0.163 209.51.188.20 FTP 60 Request: LIST

Frame 88: Packet, 105 bytes on wire (840 bits), 105 bytes captured (840 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE},
Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4, Src: 209.51.188.20, Dst: 192.168.0.163
Transmission Control Protocol, Src Port: 21, Dst Port: 55828, Seq: 1198, Ack: 65, Len: 51
File Transfer Protocol (FTP)
  227 Entering Passive Mode (209,51,188,20,83,100).\r\n
    Response code: Entering Passive Mode (227)
    Response arg: Entering Passive Mode (209,51,188,20,83,100).
    Passive IP address: 209.51.188.20
    Passive port: 21348
[Current working directory: /gnu+linux-distros]
[Command: LIST]
[Command frame: 89]

```

Рисунок 18 – Ответ сервера с IP и портом для пассивного режима

Вопрос 3: Чем отличаются пакеты FTP от FTP-DATA?

Пакеты FTP и FTP-DATA – это два разных потока данных, которые служат разным целям и передаются по разным TCP-соединениям.

- Пакеты FTP (Управляющее соединение): Содержат команды от клиента к серверу (например, USER, PASS, PASV, RETR filename) и ответы сервера в виде кодов состояния (например, 230 Login successful, 227 Entering Passive Mode). Эти данные передаются в простом текстовом виде и служат для управления сессией. Это соединение остается активным на протяжении всей сессии.
- Пакеты FTP-DATA (Соединение для данных): Содержат исключительно "сырые" данные – то есть, содержимое передаваемого файла. В этих пакетах нет никаких команд или кодов состояния. Это соединение устанавливается только на время передачи одного файла и сразу после этого закрывается.

123	8.192395	209.51.188.20	192.168.0.163	FTP	113	Response: 257 "/gnu/linux-distros/ututo-e" is the current directory
124	8.192605	192.168.0.163	209.51.188.20	FTP	60	Request: PASV
125	8.359852	209.51.188.20	192.168.0.163	FTP	104	Response: 227 Entering Passive Mode (209,51,188,20,99,31).
126	8.360144	192.168.0.163	209.51.188.20	FTP	60	Request: LIST
133	8.655030	209.51.188.20	192.168.0.163	FTP	93	Response: 150 Here comes the directory listing.
134	8.655030	209.51.188.20	192.168.0.163	FTP-DA...	186	FTP Data: 132 bytes (PASV) (LIST)
140	8.826781	209.51.188.20	192.168.0.163	FTP	78	Response: 226 Directory send OK.
144	9.927674	192.168.0.163	209.51.188.20	FTP	68	Request: CWD /mirrors
146	10.066673	209.51.188.20	192.168.0.163	FTP	91	Response: 250 Directory successfully changed.
147	10.066885	192.168.0.163	209.51.188.20	FTP	59	Request: PWD
151	10.217684	209.51.188.20	192.168.0.163	FTP	95	Response: 257 "/mirrors" is the current directory
152	10.217955	192.168.0.163	209.51.188.20	FTP	60	Request: PASV
154	10.364917	209.51.188.20	192.168.0.163	FTP	106	Response: 227 Entering Passive Mode (209,51,188,20,103,182).
155	10.365205	192.168.0.163	209.51.188.20	FTP	60	Request: LIST
160	10.662180	209.51.188.20	192.168.0.163	FTP	93	Response: 150 Here comes the directory listing.
161	10.662180	209.51.188.20	192.168.0.163	FTP-DA...	121	FTP Data: 67 bytes (PASV) (LIST)
167	10.823582	209.51.188.20	192.168.0.163	FTP	78	Response: 226 Directory send OK.
198	12.939789	209.51.188.20	192.168.0.163	FTP	81	Response: 220 GNU FTP server ready.
199	12.941491	192.168.0.163	209.51.188.20	FTP	70	Request: USER anonymous
201	13.095135	209.51.188.20	192.168.0.163	FTP	93	Response: 230-NOTICE (Updated October 15 2021):
202	13.095135	209.51.188.20	192.168.0.163	FTP	126	Response: 230-
203	13.095135	209.51.188.20	192.168.0.163	FTP	122	Response: 230-we strongly encourage you to change them to use HTTPS instead.
204	13.095135	209.51.188.20	192.168.0.163	FTP	60	Response: 230-
205	13.095135	209.51.188.20	192.168.0.163	FTP	121	Response: 230-Eventually we hope to shut down FTP protocol access, but plan
206	13.095135	209.51.188.20	192.168.0.163	FTP	135	Response: 230-to give notice here and other places for several months ahead
207	13.095135	209.51.188.20	192.168.0.163	FTP	147	Response: 230-
208	13.095135	209.51.188.20	192.168.0.163	FTP	112	Response: 230-site is subject to the following legal notice:
209	13.095135	209.51.188.20	192.168.0.163	FTP	201	[TCP Previous segment not captured] Response: 230- which, together with ob
211	13.239334	209.51.188.20	192.168.0.163	FTP	370	Response: 230- States under License Exception "TSU" pursuant to 15 C.F.R.
213	13.271094	209.51.188.20	192.168.0.163	FTP	124	[TCP Out-Of-Order] Response: 230- This site includes publicly available en
215	13.273887	192.168.0.163	209.51.188.20	FTP	86	Request: CWD /gnu/linux-distros/ututo-e
216	13.423789	209.51.188.20	192.168.0.163	FTP	91	Response: 250 Directory successfully changed.
217	13.424687	192.168.0.163	209.51.188.20	FTP	62	Request: TYPE A
218	13.577242	209.51.188.20	192.168.0.163	FTP	84	Response: 200 Switching to ASCII mode.
219	13.577489	192.168.0.163	209.51.188.20	FTP	60	Request: PASV
220	13.725740	209.51.188.20	192.168.0.163	FTP	105	Response: 227 Entering Passive Mode (209,51,188,20,106,81).
221	13.726123	192.168.0.163	209.51.188.20	FTP	67	Request: RETR README
229	14.031181	209.51.188.20	192.168.0.163	FTP	118	Response: 150 Opening BINARY mode data connection for README (48 bytes).
230	14.034216	209.51.188.20	192.168.0.163	FTP-DA...	102	FTP Data: 48 bytes (PASV) (RETR README)
238	14.182095	209.51.188.20	192.168.0.163	FTP	78	Response: 226 Transfer complete.

Рисунок 19 – Разделение трафика на FTP и FTP-DATA

1.8 Анализ DHCP-трафика

1.8.1 Описание выполненных действий

Для анализа протокола DHCP был использован Wireshark с фильтром bootp (на сегодняшний день можно использовать dhcp). В командной строке с правами администратора была дважды выполнена последовательность команд `ipconfig /release` (для сброса текущего IP-адреса) и `ipconfig /renew` (для получения нового IP-адреса от DHCP-сервера).

1.8.2 Временная диаграмма обмена (DORA)

Ниже представлена временная диаграмма обмена четырьмя основными DHCP-пакетами (Discover, Offer, Request, ACK), полученная в результате выполнения первой команды `ipconfig /renew`.

Весь обмен происходит по протоколу UDP: клиент использует порт 68, а сервер – порт 67.

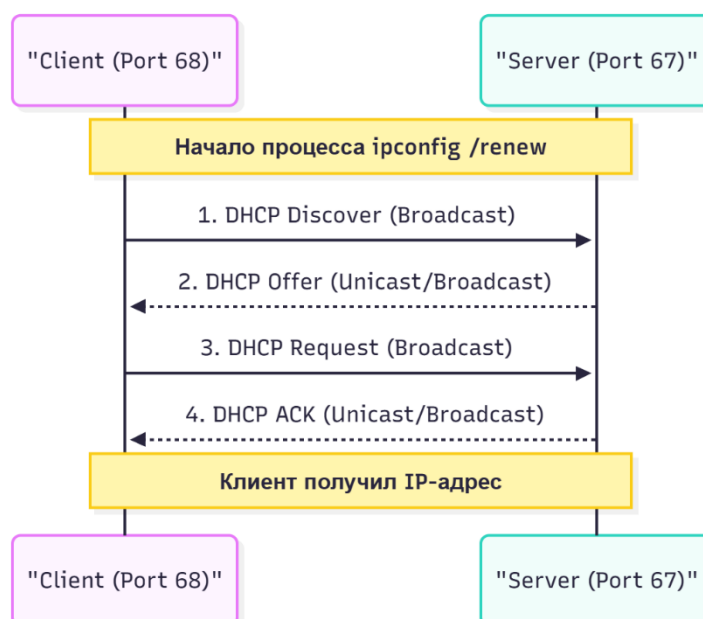


Рисунок 20 – Диаграмма процесса получения IP-адреса (DHCP DORA)

1.8.3 Ответы на вопросы задания

Вопрос 1: Чем различаются пакеты «DHCP Discover» и «DHCP Request»?

Оба пакета отправляются клиентом, но на разных этапах и с разной целью:

DHCP Discover: Это первый шаг. Клиент еще не знает ни о каких DHCP-серверах. Он просит у всей сети: «Есть тут хоть один DHCP-сервер? Мне нужен адрес!». В этом пакете клиент не запрашивает конкретный IP, а просто ищет сервер.

DHCP Request: Это третий шаг. Клиент уже получил одно или несколько предложений (Offer) от серверов. Теперь он официально запрашивает конкретный IP-адрес у конкретного сервера (чье предложение он выбрал). В этом пакете есть специальная опция (Option 50, «Requested IP Address»), где указан желаемый IP.

Вопрос 2: Как и почему менялись MAC- и IP-адреса источника и назначения в переданных DHCP-пакетах.

Адресация в DHCP меняется, так как у клиента изначально нет IP-адреса.

Discover и Request (от Клиента):

- IP Src: 0.0.0.0 (У меня еще нет адреса).
- IP Dst: 255.255.255.255 (Широковещательный, всем серверам).
- MAC Src: Мой MAC.
- MAC Dst: ff:ff:ff:ff:ff:ff (Всем в локальной сети).

Offer и ACK (от Сервера):

- IP Src: IP-адрес сервера.
- IP Dst: Либо широковещательный 255.255.255.255, либо будущий IP клиента (зависит от реализации).
- MAC Src: MAC сервера.
- MAC Dst: Мой MAC (сервер узнал его из моего запроса).

Клиент идентифицирует себя не по IP, а по своему MAC-адресу, который передается внутри поля Client MAC address самого протокола DHCP.

bootp						
No.	Time	Source	Destination	Protocol	Length	Info
251	4.277916	192.168.0.163	192.168.0.1	DHCP	342	DHCP Release - Transaction ID 0x3ced182e
410	11.834223	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x34b5e65f
411	11.839659	192.168.0.1	192.168.0.163	DHCP	590	DHCP Offer - Transaction ID 0x34b5e65f
412	11.840540	0.0.0.0	255.255.255.255	DHCP	364	DHCP Request - Transaction ID 0x34b5e65f
413	11.850645	192.168.0.1	192.168.0.163	DHCP	590	DHCP ACK - Transaction ID 0x34b5e65f
1598	29.881807	192.168.0.163	192.168.0.1	DHCP	342	DHCP Release - Transaction ID 0x1e220c8a
1684	34.345840	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x793aaaae
1685	34.351021	192.168.0.1	192.168.0.163	DHCP	590	DHCP Offer - Transaction ID 0x793aaaae
1686	34.351920	0.0.0.0	255.255.255.255	DHCP	364	DHCP Request - Transaction ID 0x793aaaae
1687	34.361575	192.168.0.1	192.168.0.163	DHCP	590	DHCP ACK - Transaction ID 0x793aaaae

Рисунок 21 – Список пакетов DHCP-обмена (DORA)

Вопрос 3: Каков IP-адрес DHCP-сервера?

IP-адрес DHCP-сервера можно увидеть в поле Source IP пакетов DHCP Offer и DHCP ACK. В моем случае IP-адрес DHCP-сервера – 192.168.0.1.

Также этот адрес передается внутри самого DHCP-сообщения в специальной опции (Option 54, «DHCP Server Identifier»).

Весь трафик оказался зашифрованным, поэтому анализ производился на основе метаданных: используемых транспортных протоколов, интенсивности и размера пакетов.

1.9.2 Ответы на вопросы задания

Вопрос 1: Чем различаются пакеты разных видов VK-трафика (текст, аудио, видео)?

Анализ трех трасс трафика показал существенные различия между видами коммуникации, в первую очередь на транспортном уровне.

Текстовые сообщения:

Трафик текстовых сообщений и служебных команд передается по протоколу TCP (инкапсулированным в TLS для шифрования). Как видно из лога vk_text, обмен с серверами VK (87.240.190.70) происходит нерегулярными «всплесками» пакетов Application Data в моменты отправки и получения сообщений. Это обусловлено необходимостью гарантированной доставки текстовой информации.

tcp.port == 443						
No.	Time	Source	Destination	Protocol	Length	Info
46	3.497356	90.156.232.15	192.168.0.163	TCP	54	443 → 23409 [ACK] Seq=1 Ack=2 Win=83 Len=0
47	3.591184	192.168.0.163	162.159.133.234	TLSv1.2	175	Application Data
48	3.602899	162.159.133.234	192.168.0.163	TCP	54	443 → 51417 [ACK] Seq=1 Ack=122 Win=16 Len=0
49	3.733277	162.159.133.234	192.168.0.163	TLSv1.2	88	Application Data
50	3.778267	192.168.0.163	162.159.133.234	TCP	54	51417 → 443 [ACK] Seq=122 Ack=35 Win=253 Len=0
51	3.851930	192.168.0.163	51.137.3.145	TCP	55	12765 → 443 [ACK] Seq=1 Ack=1 Win=255 Len=1
52	3.856938	192.168.0.163	87.240.190.70	TLSv1.2	318	Application Data
53	3.867375	192.168.0.163	172.211.159.152	TCP	55	49821 → 443 [ACK] Seq=1 Ack=1 Win=255 Len=1
54	3.888374	51.137.3.145	192.168.0.163	TCP	66	443 → 12765 [ACK] Seq=1 Ack=2 Win=63 Len=0 SLE=1 SRE=2
55	3.903265	172.211.159.152	192.168.0.163	TCP	66	443 → 49821 [ACK] Seq=1 Ack=2 Win=63 Len=0 SLE=1 SRE=2
56	3.908401	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=1335 Ack=1112 Win=31 Len=0
57	4.312918	162.159.133.234	192.168.0.163	TLSv1.2	112	Application Data
58	4.367577	192.168.0.163	162.159.133.234	TCP	54	51417 → 443 [ACK] Seq=122 Ack=93 Win=252 Len=0
59	5.051932	87.240.190.70	192.168.0.163	TLSv1.2	620	Application Data
60	5.055177	192.168.0.163	87.240.190.70	TLSv1.2	171	Application Data
61	5.055246	192.168.0.163	87.240.190.70	TLSv1.2	178	Application Data
62	5.061882	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=1901 Ack=1229 Win=31 Len=0
63	5.061882	87.240.190.70	192.168.0.163	TLSv1.2	89	Application Data
64	5.108235	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=1936 Ack=1353 Win=31 Len=0
65	5.113052	192.168.0.163	87.240.190.70	TCP	54	23407 → 443 [ACK] Seq=1353 Ack=1936 Win=1023 Len=0
69	6.750461	192.168.0.163	169.150.223.48	TCP	55	51416 → 443 [ACK] Seq=1 Ack=1 Win=255 Len=1
70	6.789733	169.150.223.48	192.168.0.163	TCP	66	443 → 51416 [ACK] Seq=1 Ack=2 Win=448 Len=0 SLE=1 SRE=2
71	7.097670	87.240.190.70	192.168.0.163	TLSv1.2	626	Application Data
72	7.098908	192.168.0.163	87.240.190.70	TLSv1.2	89	Application Data
73	7.100789	192.168.0.163	87.240.190.70	TLSv1.2	170	Application Data
74	7.100846	192.168.0.163	87.240.190.70	TLSv1.2	178	Application Data
75	7.105837	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=2508 Ack=1388 Win=31 Len=0
76	7.105837	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=2508 Ack=1504 Win=31 Len=0
77	7.107876	87.240.190.70	192.168.0.163	TLSv1.2	89	Application Data
78	7.156072	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=2543 Ack=1628 Win=31 Len=0
79	7.158251	192.168.0.163	87.240.190.70	TCP	54	23407 → 443 [ACK] Seq=1628 Ack=2543 Win=1021 Len=0
80	9.914213	192.168.0.163	95.213.56.1	TCP	55	23406 → 443 [ACK] Seq=1 Ack=1 Win=255 Len=1
81	9.921591	95.213.56.1	192.168.0.163	TCP	66	443 → 23406 [ACK] Seq=1 Ack=2 Win=21 Len=0 SLE=1 SRE=2
82	10.615458	192.168.0.163	169.150.223.48	TLSv1.2	102	Application Data
83	10.615525	192.168.0.163	169.150.223.48	TLSv1.2	93	Application Data
84	10.633324	87.240.190.70	192.168.0.163	TLSv1.2	630	Application Data
85	10.636771	192.168.0.163	87.240.190.70	TLSv1.2	170	Application Data
86	10.636847	192.168.0.163	87.240.190.70	TLSv1.2	178	Application Data
87	10.643386	87.240.190.70	192.168.0.163	TCP	54	443 → 23407 [ACK] Seq=3119 Ack=1744 Win=31 Len=0

Рисунок 23 – Нерегулярный характер TCP-трафика текстовых сообщений

Аудиозвонок: Трафик аудиозвонка разделяется на две части. Сигнальная часть (установка вызова) использует TCP/TLS. Сам же медиа-поток (голос) передается по протоколу QUIC, который работает поверх UDP. В логе vk_audio виден интенсивный,

непрерывный поток QUIC-пакетов между моим компьютером и медиа-сервером (5.61.23.5). UDP используется, так как для голоса приоритетом является скорость доставки, а не 100% надежность.

No.	Time	Source	Destination	Protocol	Length	Info
267	13.262486	192.168.0.163	93.186.227.135	QUIC	1292	Initial, DCID=eca0771dca0f5e8b, PKN: 1, CRYPTO
268	13.262573	192.168.0.163	93.186.227.135	QUIC	1292	Initial, DCID=eca0771dca0f5e8b, PKN: 2, PADDING, PING, CRYPTO, PING, CRYPTO, PING, CRYPTO, PADDING, CRYPTO,
271	13.270553	93.186.227.135	192.168.0.163	QUIC	151	Retry, SCID=79d576e769acff7a8185ba8bb67cd06cb7c1a7b1
272	13.270553	93.186.227.135	192.168.0.163	QUIC	151	Retry, SCID=138978c993019033ccb607650704f9e91fd2d820
280	13.270994	192.168.0.163	93.186.227.135	QUIC	1292	Initial, DCID=79d576e769acff7a8185ba8bb67cd06cb7c1a7b1, PKN: 3, CRYPTO
281	13.271051	192.168.0.163	93.186.227.135	QUIC	1292	Initial, DCID=79d576e769acff7a8185ba8bb67cd06cb7c1a7b1, PKN: 4, PADDING, CRYPTO, CRYPTO, PING, CRYPTO, CRY
283	13.278319	93.186.227.135	192.168.0.163	QUIC	93	Initial, SCID=000000000000b01f94817f40669fb1263fdcae, PKN: 0, ACK
284	13.278319	93.186.227.135	192.168.0.163	QUIC	1242	Handshake, SCID=000000000000b01f94817f40669fb1263fdcae
285	13.279072	192.168.0.163	93.186.227.135	QUIC	215	Protected Payload (KP0), DCID=000000000000b01f94817f40669fb1263fdcae
286	13.288310	93.186.227.135	192.168.0.163	QUIC	248	Protected Payload (KP0)
287	13.325102	192.168.0.163	93.186.227.135	QUIC	86	Protected Payload (KP0), DCID=000000000000b01f94817f40669fb1263fdcae
289	13.330664	93.186.227.135	192.168.0.163	QUIC	63	Protected Payload (KP0)
290	13.330664	93.186.227.135	192.168.0.163	QUIC	63	Protected Payload (KP0)
291	13.330984	192.168.0.163	93.186.227.135	QUIC	85	Protected Payload (KP0), DCID=000000000000b01f94817f40669fb1263fdcae
292	13.388784	93.186.227.135	192.168.0.163	QUIC	1514	Protected Payload (KP0)
293	13.419172	192.168.0.163	93.186.227.135	QUIC	86	Protected Payload (KP0), DCID=000000000000b01f94817f40669fb1263fdcae

Рисунок 24 – Непрерывный поток QUIC-пакетов при аудиозвонке

Видеозвонок: В данном эксперименте приложение использовало резервный механизм передачи медиа-потока через TCP (инкапсулированный в TLSv1.3). Вместо UDP, наблюдался очень интенсивный и непрерывный поток TCP-пакетов значительно большего размера (часто достигающих максимального размера сегмента, ~1460 байт). Несмотря на использование TCP, поведенческие характеристики (высокая интенсивность и большой размер пакетов) однозначно указывают на передачу видео.

No.	Time	Source	Destination	Protocol	Length	Info
44	1.306123	192.168.0.163	5.61.23.5	TCP	66	1861 → 23433 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
45	1.321621	5.61.23.5	192.168.0.163	TCP	66	23433 → 1861 [SYN, ACK] Seq=0 Ack=1 Win=32120 Len=0 MSS=1440 SACK_PERM WS=1
46	1.321755	192.168.0.163	5.61.23.5	TCP	54	1861 → 23433 [ACK] Seq=1 Ack=1 Win=65280 Len=0
47	1.322344	192.168.0.163	5.61.23.5	TCP	1494	1861 → 23433 [ACK] Seq=1 Ack=1 Win=65280 Len=1440 [TCP PDU reassembled in 51]
48	1.322344	192.168.0.163	5.61.23.5	TCP	1494	1861 → 23433 [ACK] Seq=1441 Ack=1 Win=65280 Len=1440 [TCP PDU reassembled in 51]
49	1.322344	192.168.0.163	5.61.23.5	TCP	1494	1861 → 23433 [ACK] Seq=2881 Ack=1 Win=65280 Len=1440 [TCP PDU reassembled in 51]
50	1.322344	192.168.0.163	5.61.23.5	TCP	1494	1861 → 23433 [ACK] Seq=4321 Ack=1 Win=65280 Len=1440 [TCP PDU reassembled in 51]
51	1.322344	192.168.0.163	5.61.23.5	TLSv1.3	139	Client Hello (SNI=videowebtrc.okcdn.ru)
52	1.337116	5.61.23.5	192.168.0.163	TCP	54	23433 → 1861 [ACK] Seq=1 Ack=1441 Win=30680 Len=0
53	1.337116	5.61.23.5	192.168.0.163	TCP	54	23433 → 1861 [ACK] Seq=1 Ack=4321 Win=30240 Len=0
54	1.337116	5.61.23.5	192.168.0.163	TCP	54	23433 → 1861 [ACK] Seq=1 Ack=5846 Win=30240 Len=0
55	1.340760	5.61.23.5	192.168.0.163	TLSv1.3	1514	Server Hello, Change Cipher Spec
56	1.340760	5.61.23.5	192.168.0.163	TCP	1514	23433 → 1861 [PSH, ACK] Seq=1461 Ack=5846 Win=30240 Len=1460 [TCP PDU reassembled in 57]
57	1.340760	5.61.23.5	192.168.0.163	TLSv1.3	1493	Application Data
58	1.340855	192.168.0.163	5.61.23.5	TCP	54	1861 → 23433 [ACK] Seq=5846 Ack=4360 Win=65280 Len=0
59	1.341141	192.168.0.163	5.61.23.5	TLSv1.3	118	Change Cipher Spec, Application Data
60	1.341336	192.168.0.163	5.61.23.5	TLSv1.3	909	Application Data
63	1.356294	5.61.23.5	192.168.0.163	TCP	1514	23433 → 1861 [ACK] Seq=4360 Ack=5910 Win=30240 Len=1460 [TCP PDU reassembled in 65]
64	1.356294	5.61.23.5	192.168.0.163	TCP	1514	23433 → 1861 [PSH, ACK] Seq=5820 Ack=5910 Win=30240 Len=1460 [TCP PDU reassembled in 65]
65	1.356294	5.61.23.5	192.168.0.163	TLSv1.3	1270	Application Data
66	1.356294	5.61.23.5	192.168.0.163	TLSv1.3	267	Application Data
67	1.356397	192.168.0.163	5.61.23.5	TCP	54	1861 → 23433 [ACK] Seq=6765 Ack=8709 Win=65280 Len=0
70	1.374422	5.61.23.5	192.168.0.163	TLSv1.3	881	Application Data
71	1.374422	5.61.23.5	192.168.0.163	TLSv1.3	263	Application Data
72	1.374520	192.168.0.163	5.61.23.5	TCP	54	1861 → 23433 [ACK] Seq=6765 Ack=9745 Win=64256 Len=0
75	1.444053	192.168.0.163	5.61.23.5	TLSv1.3	162	Application Data
76	1.444146	192.168.0.163	5.61.23.5	TLSv1.3	121	Application Data
77	1.461011	5.61.23.5	192.168.0.163	TCP	54	23433 → 1861 [ACK] Seq=9745 Ack=6940 Win=30240 Len=0

Рисунок 25 – Видеозвонок в режиме «TCP Fallback»:

непрерывный поток TCP-пакетов большого размера

Вопрос 2: Какой Wireshark-фильтр следует использовать для независимой идентификации VK-трафика разных видов (текст, аудио, видео)?

Поскольку весь трафик VK зашифрован, невозможно создать фильтр на основе содержимого пакетов. Однако, основываясь на анализе характеристик трафика, можно

составить эффективные фильтры на основе метаданных: IP-адресов и используемых транспортных протоколов. Процесс состоит из двух шагов.

Шаг 1: Определение IP-адресов серверов VK

В первую очередь необходимо определить IP-адреса, с которыми взаимодействует приложение. Это можно сделать с помощью инструмента Статистика -> Конечные точки -> IPv4. В ходе моих экспериментов были идентифицированы следующие типы серверов:

- Серверы для текста и сигнализации: 87.240.190.70, 185.16.148.52 и др. (используют TCP).
- Медиа-серверы для звонков: 5.61.23.5 (videowebrtc.okcdn.ru) и др. (используют QUIC/UDP или TCP в качестве резерва).

Шаг 2: Составление фильтров для разных видов трафика

Фильтр для текстовых сообщений и сигнализации:

Поскольку и текстовые сообщения, и управляющие команды требуют надежной доставки, они используют протокол TCP.

`tcp and (ip.addr == 87.240.190.70 or ip.addr == 185.16.148.52)`

Этот фильтр изолирует зашифрованный (TLSv1.2) трафик, который передается по TCP к известным серверам VK, отвечающим за сообщения.

Фильтр для медиа-трафика (аудио и видео):

Для передачи голоса и видео в реальном времени приоритетом является скорость, а не надежность, поэтому используется UDP. В моем случае это был протокол QUIC.

`quic`

Этот фильтр эффективно отсекает весь медиа-трафик звонков, поскольку и аудио, и (в штатном режиме) видео используют QUIC. Он позволяет увидеть непрерывный поток пакетов, характерный для медиа.

Разделение аудио- и видео-трафика:

Разделить аудио и видео только с помощью фильтра отображения практически невозможно, если оба используют один и тот же протокол (например, QUIC). Однако их можно различить косвенным анализом или с помощью уникальной ситуации, возникшей в моем эксперименте:

- В моем конкретном случае аудио-трафик использовал QUIC, а видео-трафик переключился на резервный механизм и использовал TCP. Поэтому их можно было разделить фильтрами:

- Аудио: `quic and ip.addr == 87.240.190.70`
- Видео: `tcp and ip.addr == 5.61.23.5`

– В общем случае для разделения аудио и видео после применения фильтра `quic` необходимо использовать инструменты анализа Wireshark:

- Визуально: по колонке `Length`. Пакеты видео-трафика в среднем значительно крупнее пакетов аудио-трафика.
- С помощью графика: Статистика -> I/O Graph. График битрейта (скорости передачи) для видеозвонка будет показывать значения в несколько раз выше, чем для аудиозвонка. Это самый надежный способ их отличить.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной учебно-исследовательской работы была достигнута поставленная цель по изучению структуры протокольных блоков данных с помощью анализатора трафика Wireshark.

Были освоены практические навыки захвата, фильтрации и анализа сетевого трафика. Проведен детальный разбор структуры кадров, пакетов и сегментов для ключевых протоколов стека TCP/IP.

В частности, были исследованы механизмы работы служебных протоколов ICMP (на примере фрагментации пакетов и трассировки маршрута), DNS (разрешение имен, типы записей) и ARP (сопоставление IP- и MAC-адресов). Проанализированы протоколы прикладного уровня HTTP (механизмы кэширования) и FTP (разделение управляющего и информационного каналов). Наглядно изучен процесс получения IP-адреса по протоколу DHCP.

Особое внимание было уделено анализу зашифрованного трафика современного мессенджера VK, где были выявлены и объяснены различия в транспортных протоколах (TCP и QUIC/UDP) и характеристиках потоков данных для текста, аудио- и видеосвязи.

Выполненная работа позволила не только закрепить теоретические знания о компьютерных сетях, но и получить глубокое практическое понимание принципов взаимодействия сетевых устройств и протоколов в реальных условиях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алиев Т.И., Соснин В.В., Шинкарук Д.Н. Компьютерные сети и телекоммуникации: задания и тесты. – СПб: Университет ИТМО – 2018. – 112 с.
2. Куроуз Дж. Ф., Росс К. В. Компьютерные сети: Нисходящий подход / пер. с англ. – 6-е изд. – М.: Эксмо, 2016. – 912 с.